Trabajo Práctico: ARMv8 en QEMU

Introducción

El trabajo práctico se basa en el uso de la herramienta QEMU para emular un microprocesador ARMv8 y el uso del debugger GDB para depurar un programa que deberán construir. El programa, escrito en lenguaje ensamblador ARMv8, deberá resolver de manera genérica laberintos representados en formato ASCII.

Condiciones

- Realizar el trabajo en grupos de exactamente *4 personas* (**deben inscribirse en el siguiente link antes del viernes 22 de mayo**).
- Tienen tiempo de entregar el laboratorio hasta **las 23:59hs** del **viernes 12 de junio** En la evaluación se va a considerar el último commit realizado antes de la fecha y hora de entrega. (No se considerarán nuevos trabajos pasado este límite de tiempo. Los trabajos entregados por cualquier otro medio al establecido se consideran desaprobados).
- La aprobación del ejercicio 1 de este trabajo es requisito obligatorio para obtener la REGULARIDAD de la materia. La aprobación de los ejercicios 1 y 2 es requisito obligatorio para obtener la PROMOCIÓN de la materia.
- No está permitido el uso de inteligencias artificiales generativas.
- No está permitido compartir código entre grupos, ni dejar el código en repositorios públicos.

Formato de entrega

Se debe trabajar en el repositorio git, creado a partir de este-classroom, usando el nombre de grupo asignado por la cátedra. En el mismo habrá dos carpetas con la siguiente denominación: *ejercicio1, ejercicio2*. Estas carpetas deben contener, todos y únicamente, los archivos necesarios para la compilación (debe hacerse un *make clean* antes del commit). Además, se debe agregar un pequeño informe, que será un archivo de texto plano que contenga los datos de los integrantes del grupo (nombre, apellido y DNI) y una descripción en unas pocas líneas (3 o 4) de la metodología usada para resolver cada ejercicio.

El archivo principal (main.s) debe contener comentarios que ayuden a comprender la manera en que solucionaron el problema.

Calificación

El trabajo de laboratorio no lleva nota. Para aprobar, los códigos deben realizar la tarea pedida, que será corroborada cargando, ensamblando y ejecutando el programa; y luego validado por inspección ocular. También se va a verificar que todos los integrantes hayan trabajado en el repositorio.

Aunque no se califica: estilo de código, simpleza, elegancia, comentarios adecuados y velocidad; si el código está muy por fuera de los parámetros aceptables, se podrá desaprobar el trabajo aunque sea funcionalmente correcto.

Ejercicio 1

Desarrollar un programa en lenguaje ensamblador ARMv8 que resuelva de manera genérica laberintos representados en formato ASCII. El programa deberá encontrar la salida de un laberinto de dimensiones N×16 utilizando una lógica de navegación autónoma.

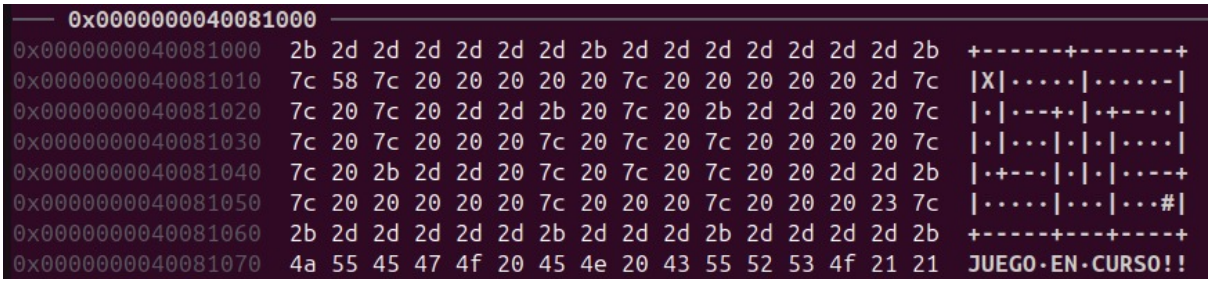
Visualización del laberinto

El código se ejecuta en el entorno de simulación QEMU+GDB, el laberinto se encuentra almacenado en memoria ram, codificado en ascii. La ventana de **Memory Watch** del GDB muestra el contenido de la memoria RAM, según el siguiente formato:

1. La Columna de Direcciones (Izquierda): Muestra la dirección de memoria dónde comienza cada fila.

2. La Columna Hexadecimal (Centro) Aquí están los los valores numéricos crudos que procesa la CPU. Cada bloque de dos dígitos representa 1 byte (8 bits).

3. La Columna ASCII (Derecha) El ASCII (American Standard Code for Information Interchange) es simplemente una tabla de traducción. La memoria almacena exclusivamente valores binarios, estos son interpretados mediante el visualizador según el estándar ASCII para mostrarnos qué símbolo representa cada número.



Nota: En este visualizador, el espacio vacío se muestra como un punto "**.**".

Especificaciones del Entorno

- **Representación del Laberinto:** Se encuentra definido en el archivo mapa.s y se cargará en la dirección de memoria fija 0x0000000040081000. Al inicio de la ejecución, el registro x0 ya contendrá dicha dirección base.
- **Dimensiones:** El laberinto tiene un ancho fijo de 16 bytes (caracteres) y un alto N variable.
- **Símbolos del Sistema:**

X	Personaje principal	0x58
	Espacio vacío	0x20
#	Salida	0x23
|	Pared vertical	0x7c

+	Esquinas	0x2b
-	Pared horizontal	0x2d

Requerimientos del Programa

1. **Navegación:** El personaje debe desplazarse únicamente por los espacios vacíos (0x20) hasta alcanzar la salida # (0x23). No se permite atravesar paredes ni esquinas.
2. **Número de pasos:** El registro x1 debe iniciarse en cero y contar la cantidad de desplazamientos (pasos) realizados por el personaje. *Nota: Los giros sobre el mismo eje no deben contarse como pasos.*
3. **Condición de Éxito:** Al alcanzar la salida (#), el programa debe escribir la cadena de caracteres "Juego finalizado" (16 bytes) en la dirección de memoria asociada a la etiqueta estado.
4. **Restricciones de Implementación:**
   - **Código:** Toda la lógica debe residir exclusivamente en la zona delimitada del archivo main.s. No es posible subdividir el código en archivos.
   - **Archivos:** No está permitido modificar los archivos Makefile, memmap ni la estructura de mapa.s.
   - **Memoria:** El tamaño total del programa (código + datos, excluyendo el laberinto) no debe exceder los 4KB.
   - **Direccionamiento:** No está permitido modificar la dirección base en memoria donde se almacena el laberinto.
5. **Inicio del personaje:** Dado que el resolvedor es genérico, el programa debe localizar primero la posición actual de la **X** dentro del laberinto antes de iniciar el movimiento.

Desplazamiento

Aunque visualmente vemos el laberinto como una matriz de N×16, la memoria RAM es un vector contiguo. Cada carácter ocupa exactamente **1 byte**. Si el laberinto comienza en la dirección base (almacenada en x0), la ubicación de cualquier celda [fila, columna] se calcula con la siguiente fórmula:

```
Dirección = DirBase + (fila×16) + columna
```

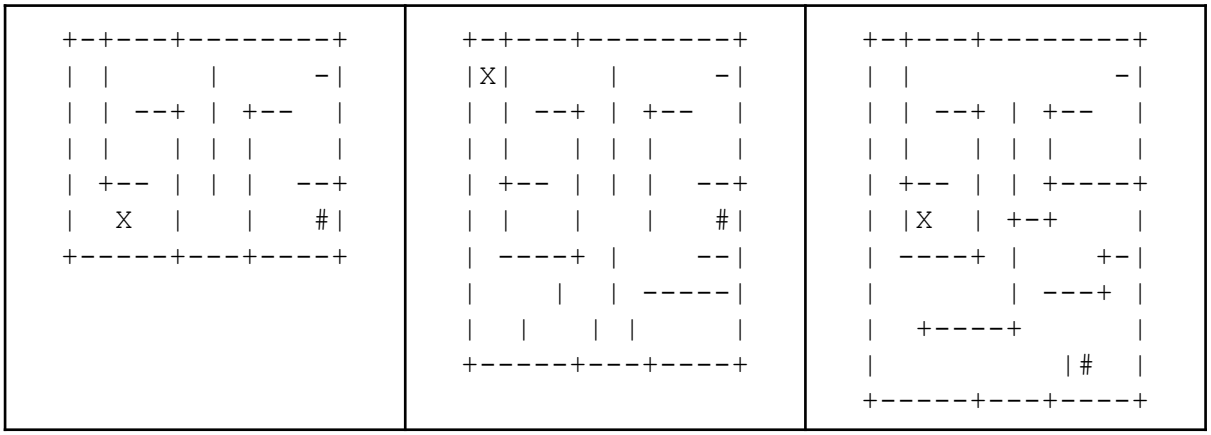
Mover al personaje consiste en un proceso de tres pasos en la memoria:

1. **Leer:** Consultar qué valor hay en la dirección de destino.
2. **Validar:** Comparar si ese valor es un camino transitable (0x20) o la salida (0x23).
3. **Escribir :** Si el movimiento es válido, debemos actualizar la memoria:
   - Escribir un espacio vacío (0x20) en la **dirección vieja**.
   - Escribir una 'X' (0x58) en la **dirección nueva**.

Algoritmo Sugerido: Regla de la Mano Izquierda

Para garantizar la resolución de cualquier laberinto (siempre que esté conectado), se sugiere implementar el método de la mano izquierda. La prioridad de movimiento en cada paso, relativa a la orientación actual del personaje, es:

1. **Girar a la izquierda:** Si hay un espacio vacío a la izquierda, girar 90° y avanzar un paso.
2. **Avanzar:** Si no puede girar a la izquierda pero el frente está despejado, avanzar un paso.
3. **Girar a la derecha:** Si no puede ir a la izquierda ni al frente, girar 90° a la derecha y avanzar un paso.
4. **Retroceder:** Si se encuentra en un callejón sin salida, dar la vuelta (180°) y avanzar.
5. **Evaluación:** El resolvedor será testeado con distintos archivos mapa.s. El algoritmo debe ser lo suficientemente genérico para resolver cualquier mapa que respete las reglas de símbolos y dimensiones mencionadas. Se muestra a continuación tres tipos de laberintos a modo de ejemplo:



Ejercicio 2

Desarrollar un solucionador de laberintos en código ASCII de dimensión Nx16 similar al resuelto en el primer ejercicio. Para ello modificar el archivo '*main.s*' que les fue provisto (a través del aula virtual) en la carpeta Laberinto.

1. Objetivo

Evolucionar el resolvedor del Ejercicio 1 para que el personaje sea capaz de recolectar todos los ítems de premio (o) y utilizar sistemas de teletransportación (@) antes de alcanzar la salida.

2. Nuevos Símbolos del Sistema

Se añaden los siguientes elementos al mapa de caracteres:

o	Bonus	0x6f
@	Portal	0x40

3. Requerimientos Adicionales

1. **Recolección obligatoria:** El personaje debe recolectar **todos** los bonus (o) presentes en el laberinto antes de dirigirse a la salida (#). Al pasar sobre un bonus, este debe reemplazarse por un espacio vacío (0x20) en la memoria.
2. **Mecánica de portales:** Los portales siempre aparecen en pares y hay un único par por laberinto. Al pisar una celda @, el personaje debe ser teletransportado inmediatamente a la coordenada de la otra celda @ existente en el mapa.
    - *Nota:* El movimiento hacia el portal cuenta como un paso en x1, pero la teletransportación en sí no suma pasos adicionales.
3. **Contador de premios:** El registro x2 debe mostrar la cantidad de premios que hay en el mapa disponibles e ir disminuyendo cuando el personaje los recolecta. La cantidad de premios es variable, el programa inicialmente debe analizar cuántos premios disponibles hay en el laberinto.
4. **Condición de salida:** Si el personaje alcanza la salida (#) sin haber recolectado todos los bonus, debe ignorarla y continuar la búsqueda. La condición de éxito solo se cumple si el contador de bonus pendientes es cero.
5. **Estado:** Al finalizar exitosamente, se debe escribir "MISION COMPLETA" en la etiqueta estado.

Laberintos ejemplo a resolver:

